Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Smart Emergency Response System

Team Name:	*Alpha*
Team member 1 (Team Lead)	*Pandey, Sambhawi* *2510380216* *2510380216@geu.ac.in* 
Team member 2	*Rawat, Atharv* *2510370380* *2510370380@geu.ac.in* 
Team member 3	*Lalotra, Aayush* *2510012608* *2510012608@geu.ac.in* 
Team member 4	*Garg, Shivam 2510386512* *2510386512@geu.ac.in* 

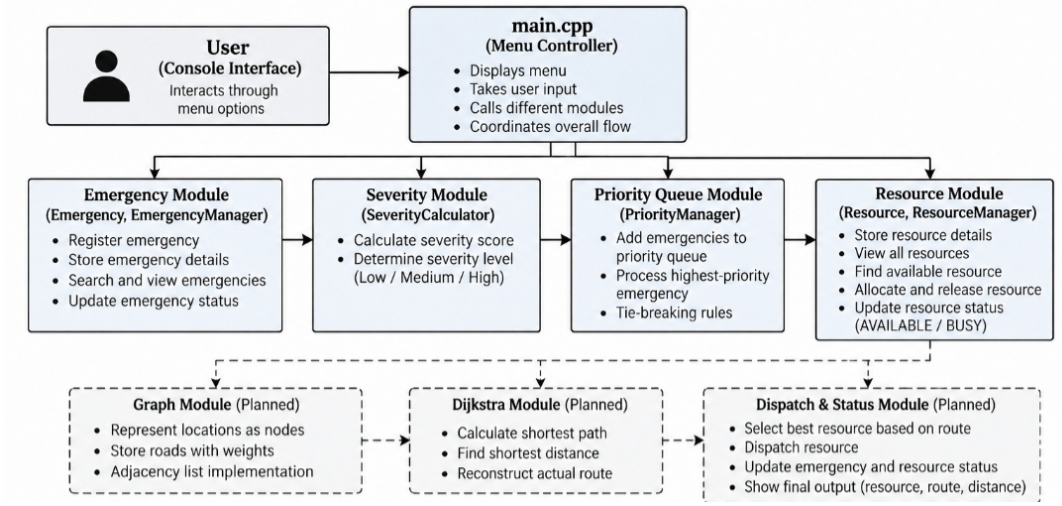
PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

The Smart Emergency Response System is a C++-based emergency management system designed to use object-oriented programming, data structures, and algorithms to register, assess, prioritize, and react to emergency situations.The system records details about an emergency, including its type, location, impacted population, urgency, and other pertinent factors. To determine the severity score and categorize the emergency depending on its degree of severity, a rule-based scoring system is employed. Higher-severity emergencies are handled first in the implemented system's Priority Queue, which is built on a Heap. Prioritized emergencies are assigned appropriate available resources using the Resource Allocation module. A weighted graph, with places as vertices and roads as weighted edges, is used to depict the road network for route management. The shortest path between the response center and the emergency site is found using Dijkstra's Algorithm.In addition to DSA principles like Priority Queue, Heap, Queue, Graph, Searching, Sorting, and shortest-path algorithms, the project illustrates useful OOP concepts like classes, objects, constructors, encapsulation, and abstraction. The fundamental C++ implementation has been completed and is operational. Integration testing, edge-case validation, debugging, output improvement, documentation, and final demonstration are the main objectives of this stage.

Updated Project Approach and Architecture

This project uses a modular Object-Oriented architecture to create the system. All of the different parts of the system are implemented in C++ and each part is a separate class or module to make it easy to develop, test, debug, and maintain. These functionalities are all developed in separate classes and modules to improve development, testing, debugging and maintenance of the system. maintain.
The current system workflow is:



The Emergency class includes an ID, type, location, affected parties, severity score, and status. The severity is calculated using a rule-based approach. The highest-priority cases are handled first using a Priority Queue/Heap. The Resource Management module assigns resources for the given situation. A weighted graph is used to represent the road network. Dijkstra's Algorithm is used to find the shortest path to the site of the given emergency.

Tasks Completed

Task Completed	Team Member
Project structure setup, module integration, main menu/controller, and integration of Emergency, Priority Queue and Resource modules.	Sambhawi Pandey
Implemented Emergency class, emergency registration, severity calculation, EmergencyManager, searching/viewing emergencies, Priority Queue/Heap and priority-based processing.	Atharv Rawat
Implemented simulated road network using Graph and adjacency list, Dijkstra shortest-path algorithm, route reconstruction, and route testing.	Aayush Lalotra
Implemented Resource class, ResourceManager, resource availability, allocation/release, resource status tracking, and resource testing.	Shivam Garg

Challenges/Roadblocks

During the development of the Smart Emergency Response System we faced the issue of bringing together diverse DSA and OOP elements into one C+ project at the same time we were to present a clean and modular structure. We had to put in extensive planning to combine modules like Emergency Management, Severity Calculation, Priority Queue, Resource Allocation, Graphs, and Dijkstra's Algorithm which had to work as a team. Also we had the challenge of weeding out a consistent rule based system for the calculation of emergency severity which was a issue as many variables had to be taken into account. We addressed this by setting forth clear scoring criteria and severity levels which in turn allowed us to classifying and prioritize emergencies effectively. Integrating the Priority Queue in with the Emergency Manager had us carefully manage emergency records which in turn was key to keeping their priority values the same as they went through the process. This in turn helped to maintain the right order in emergency response. As for working with many C+ files we had our work cut out. We had to pay attention to the header and source files, the classes, the constructors and their relationships in order to avoid compile and link errors. In order to make the integration go more smoothly each module was developed and tested out in isolation before brought together into the full system. Also we looked at the Weighted Graph and Dijkstra's algorithm implementation which had us dealing with vertices, edge weights, distance calculations and route selection. The core implementation is now up and running. In the present work we report on the evaluation of the full workflow and also address edge cases which include emergency situations of equal priority, unavailable resources, invalid input from users, and locations which are inaccessible via the present road network. We will be testing and debugging these issues in order to increase the system's reliability.

Tasks Pending

Task Pending	Team Member (to complete the task)
Integrate Graph/Dijkstra with the existing system, complete the end-to-end workflow, final integration testing, documentation and final UI/menu refinement.	Sambhawi Pandey
Integrate Graph/Dijkstra with the main system, connect resource location → emergency location routing, complete searching/sorting requirements and route edge-case testing.	Aayush Lalotra
Integrate resource selection with Graph/Dijkstra, select the suitable resource based on route/distance, complete dispatch/status lifecycle and resource-conflict/input-validation testing.	Shivam Garg
Strengthen input validation, ensure priority queue processes only appropriate pending emergencies, and complete additional priority/edge-case testing.	Atharv Rawat.

Project Outcome/Deliverables

The final result of the project is a working Smart Emergency Response System written in C++ with examples of how DSA and OOP can be used in ER. The present implementation consists of:

1. Quick register and manage emergency record.
2. Rule based severity assessment and classification.
3. Emergency priority handling based on Priority Queue/Heap.
4. Distribution of resources for high priority emergencies.
5. Road network as Weighted graph.
6. Calculation of the shortest path by Dijkstra's algorithm.
7. Search and sort feature, if applicable.
8. Modular implementation of C++ by means of classes, objects, constructors, encapsulation and abstraction.
9. An operational integrated core implementation.
10. The last milestone is the final testing and validation documentation demonstration and presentation.

Progress Overview

The project is currently in the implementation, integration, and testing phase. the have completed the initial project structure and the C++ compilation setup. The core system has been built, which includes the Emergency class, the emergency registration, the calculation of severity, the Emergency Manager, Priority Queue/Heap, Resource Allocation, weighted Graph, Dijkstra-based route.
Current progress according to the planned roadmap is:
Phase 1 – Project Setup: Completed.
Phase 2 – Emergency Registration and Emergency Class: Completed.
Phase 3 – Severity Calculation: Completed.
Phase 4 – Emergency Manager: Completed.
Phase 5 – Priority Queue/Heap: Completed.
Phase 6 – Resource Allocation: Implemented.
Phase 7 – Weighted Graph: Implemented.
Phase 8 – Dijkstra's Shortest Path: Implemented.
The main C++ program is working correctly. The rest of the work consists of doing the whole system testing, testing on the edge cases debugging putting on the finishing touches, making the output and user interaction more friendly, finishing the documentation and preparing for the final demo.

Codebase Information

The Smart Emergency Response System has been put together with C+ and it has a modular, object oriented structure. Our project is organized into header files, source files, test files, and separate data folders to improve upon issues of code readability, reusability, and maintainability. Main.cpp provides the menu driven interface and in that we also include the which coordinates the system modules. In the include directory we have our header files that present our class and interface definitions and in the src directory we have the implementations. Also in the include directory we have the Emergency class which includes emergency info such as type, location, severity, priority, and status. We also have the SeverityCalculator class that reports on severity of emergencies via a rules based approach. In addition we have the EmergencyManager for the registration, search, display of emergency records, and also to update their status. Also the PriorityManager uses the C+ STL priority queue to set priority of the emergencies based on severity and other factors. The Resource class is which of response resources and the ResourceManager which is responsible for resource availability, allocation, display and release. In the tests directory we have programs which validate module function. We use OOP concepts which include classes, objects, constructors, encapsulation and abstration also we implement DSA concepts of vectors, heaps, and priority queues. We did graph representation and put in Dijkstra's algorithm for shortest route calculation. At present we have emergency registration, severity calculation, priority based processing and basic resource management included. For the go ahead plan we have graph integration, route calculation, input validation, dispatch functionally, full scale testing and finally documentation.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Emergency Registration Test	Pass	Verified creation and storage of emergency records.
Severity Calculation Test	Pass	Tested different emergency conditions and severity scores.
Emergency Manager Test	Pass	Tested storing, displaying, and managing emergency records.
Priority Queue Test	Pass	Verified that higher-priority emergencies are processed first.

Deliverables Progress

Emergency Registration and Emergency Class – Completed.
Severity Calculation Module – Completed.
Emergency Manager – Completed.
Priority Queue/Heap – Completed.
Resource Allocation – Implemented.
Weighted Graph Representation – Implemented.
Dijkstra's Shortest Path – Implemented.
Core System Integration – Completed and running.
Testing and Validation – In Progress.
Edge-Case Testing and Debugging – In Progress.
Final Documentation – In Progress.
Final Demonstration and Presentation – Pending.
The project is currently focused on validating the complete emergency-response workflow, improving the system based on test results, and preparing the final documentation and demonstration.